



PRODUCTION INCIDENTS

— EVERY —

DEVOPS ENGINEER SHOULD KNOW



REAL INCIDENTS. REAL IMPACT. REAL LEARNING.

- INCIDENT-DRIVEN LEARNING
- FROM SYMPTOM TO SOLUTION
- LINUX • NETWORKING • DOCKER • KUBERNETES
- DATABASES • CI/CD • OBSERVABILITY
- PRACTICAL • HANDS-ON • PRODUCTION FOCUSED

UNDERSTAND. TROUBLESHOOT. FIX. PREVENT.

PRODUCTION INCIDENT FUNDAMENTALS



WHAT COUNTS AS A PRODUCTION INCIDENT?

- Any issue that affects users, services, systems, or business operations in production.
- It causes degradation, unavailability, errors, data issues, or user impact.
- If real users feel it, it's an incident.



INCIDENT VS ALERT VS PROBLEM

TERM	WHAT IT MEANS	EXAMPLE
ALERT 	An automated notification that something might be wrong.	CPU > 90% for 5 minutes
INCIDENT 	A confirmed issue that is affecting users or systems.	Website is slow and customers cannot checkout
PROBLEM 	The underlying root cause of one or more incidents.	Memory leak in application causing crashes



SEVERITY & BUSINESS IMPACT

SEVERITY	IMPACT	EXAMPLE
SEV 1 Critical (Major Outage)	Service down for most or all users. Business critical.	App completely down. No access.
SEV 2 High (Major Degradation)	Major features not working. High user impact.	Checkout failing for many users.
SEV 3 Medium (Minor Impact)	Some features impacted. Workaround exists.	Reports slow but system is still usable.
SEV 4 Low (Minimal Impact)	Minimal or no user impact.	Cosmetic issue or documentation problem.



THE FIRST RULE



UNDERSTAND IMPACT BEFORE CHANGING ANYTHING.

- Do not restart anything blindly.
- Do not scale, modify, or deploy without understanding the impact.
- Your first job is to protect users and the business.
- Collect facts. Then act.



BASIC INCIDENT WORKFLOW



WHAT JUNIOR ENGINEERS SHOULD DO WHEN AN INCIDENT BEGINS

- Stay calm. Incidents are normal.
- Acknowledge the alert or report quickly.
- Check the status page, dashboards, and recent deployments.
- Join the incident channel and listen.
- Gather facts. Don't guess.
- Ask: What is failing? Who is affected? Since when? What changed?
- Share updates clearly and often. Keep communication professional.
- Escalate if the issue is beyond you.
- Follow the runbook if available.
- Document what you did.
- Learn from every incident.



REMEMBER

INCIDENT RESPONSE IS NOT ABOUT BEING THE FASTEST. IT IS ABOUT BEING THE MOST EFFECTIVE.

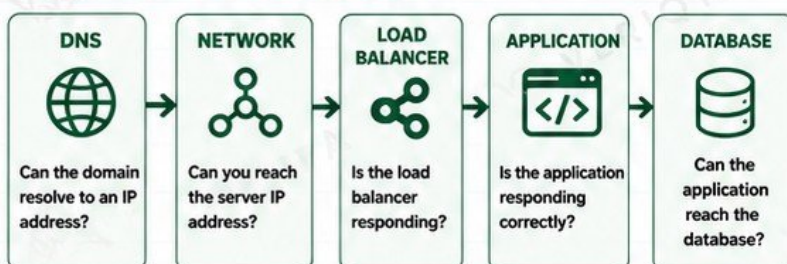
WEBSITE IS COMPLETELY DOWN



USERS REPORT THE APPLICATION IS UNAVAILABLE.



START FROM THE OUTSIDE AND WORK INWARD



TROUBLESHOOTING TOOLS YOU WILL USE

TOOL	WHAT IT DOES
curl	Checks if a service (HTTP/HTTPS) is reachable and returns a response.
ping	Checks basic network connectivity to a host.
dig	Checks DNS resolution for a domain name.
ss	Checks listening ports and network connections on a server.



STEP-BY-STEP TROUBLESHOOTING FLOW

1	CHECK DNS	<code>dig yourdomain.com +short</code>	If no IP or wrong IP → DNS problem. Fix DNS records or nameservers.
2	CHECK NETWORK	<code>ping <server-ip></code>	If unreachable → Network issue. Check routing, firewall, security groups.
3	CHECK HTTP/HTTPS REACHABILITY	<code>curl -I https://yourdomain.com</code>	If timeout or connection error → Network or firewall issue.
4	CHECK LOAD BALANCER	<code>curl -I http://<lb-dns-name></code>	If LB not responding → LB or its listeners may be down.
5	CHECK APPLICATION	<code>curl -I http://<app-ip-or-service></code>	If 5xx errors → Application problem. Check app logs and recent changes.
6	CHECK DATABASE	Check app logs / run DB connectivity test from the app server.	If DB connection fails → DB issue, credentials, or network problem.
7	VERIFY END-TO-END	<code>curl -I https://yourdomain.com</code>	You should get a 200 OK response. Confirm the site works for users.



DISTINGUISHING FAILURES



NETWORK FAILURE

- ping fails
- curl times out
- Connection refused
- No response at all

→ Problem is before the application. Focus on DNS, network, firewall, or load balancer.



APPLICATION FAILURE

- ping works
- curl connects but returns 5xx errors
- Slow or incorrect responses

→ Network is fine. Problem is in the application, code, config, or database.



AVOID RANDOMLY RESTARTING SERVICES

- Restarting without understanding can hide the real problem.
- It can cause data loss, more downtime, or worse issues.
- Gather evidence first. Then take the right action.
- Your goal is to fix the cause, not just make alerts disappear.



JUNIOR ENGINEER TIP

Always follow the flow from outside to inside. This saves time, reduces panic, and helps you find the real cause.

CPU USAGE SUDDENLY REACHES 100%



Identify the cause. Stop the pain. Fix the root problem.



1. SYMPTOMS OF CPU SATURATION

- High CPU usage (near 100%) for a long time.
- System becomes slow or unresponsive.
- Applications time out or return errors.
- High load average.
- Queues build up (requests, jobs, tasks).



2. LOAD AVERAGE VS CPU UTILIZATION

CONCEPT	MEANS	HOW TO INTERPRET
CPU UTILIZATION (top %CPU)	How busy the CPU is right now.	100% means CPU is fully busy. On multi-core, 100% means one core fully busy.
LOAD AVERAGE (uptime)	Average number of processes waiting for CPU or I/O.	If load > number of CPU cores for a long time → system is overloaded.



3. KEY COMMANDS

COMMAND	WHAT IT SHOWS	EXAMPLE
uptime	Load average and running tasks.	\$ uptime 14:25:10 up 5 days, 2 users, load average: 6.21, 5.87, 5.32
top	Real-time processes sorted by CPU usage.	\$ top %CPU %MEM COMMAND 98.7 1.2 python 45.3 0.5 nginx 20.1 0.3 java
ps	Snapshot of processes.	\$ ps -eo pid,ppid,%cpu,%mem,cmd --sort=-%cpu head PID PPID %CPU %MEM CMD 1234 1 98.7 1.2 python app.py 2345 1234 20.1 0.3 worker.sh 3456 1 5.2 0.2 systemd-journal



4. FINDING THE PROCESS CONSUMING CPU

- 1 Use top and press P to sort by %CPU.
- 2 Note the PID of the top process.
- 3 Inspect the process:
`ps -p <PID> -o pid,ppid,%cpu,%mem,cmd`
- 4 See its full command and open files:
`ls -l /proc/<PID>/fd`



5. COMMON CAUSES

- **Traffic spikes** Sudden increase in users or requests.
- **Runaway processes** A process consuming CPU in an infinite loop.
- **Loops and bugs** Poor code, retries without limits, tight loops.
- **Background jobs** Scheduled jobs overlapping or misconfigured.
- **Fork bombs** Processes creating too many child processes.
- **Resource starvation** CPU contention due to too many tasks.



6. RESTARTING MAY ONLY HIDE THE ROOT CAUSE

- Restarting kills the process, but the cause remains.
- The same problem will return.
- You lose important context and logs.
- It may interrupt other healthy services.
- Always understand why before you restart.



JUNIOR ENGINEER TIP

Always answer these questions:

- What changed?
- When did it start?
- Which process is responsible?
- Why is that process using CPU?
- How can I fix it without causing more impact?



7. WHAT YOU SHOULD DO (STEP-BY-STEP)



REMEMBER

High CPU is a symptom, not always the disease.
Find the cause, fix it, and prevent it from happening again.



veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

SERVER MEMORY IS EXHAUSTED



High memory usage slows everything down or crashes your server.



1. UNDERSTANDING MEMORY

- **RAM:** Physical memory (fast but limited).
- **Available Memory:** What can be used right now without swapping.
- **Swap:** Disk space used when RAM is full (much slower).
- When RAM is full and swap is also full, the system becomes unstable.



2. KEY COMMANDS

COMMAND	WHAT IT SHOWS	EXAMPLE OUTPUT (SHORTENED)
<code>free -h</code>	Summary of RAM and swap usage.	<pre>\$ free -h total used free shared buff/cache available Mem: 7.8Gi 7.2Gi 200Mi 88Mi 400Mi 300Mi Swap: 2.0Gi 1.6Gi 400Mi</pre>
<code>top</code>	Real-time processes sorted by memory or CPU.	<pre>\$ top (press M for memory sort) PID USER %MEM RES COMMAND 2345 www 23.1 1.8g node 1876 mysql 18.7 1.5g mysqld 998 root 10.2 820m java</pre>
<code>ps</code>	Snapshot of processes. (Use with sort)	<pre>\$ ps -eo pid,ppid,%mem,cmd --sort=-%mem head PID PPID %MEM CMD 2345 1234 23.1 node server.js 1876 987 18.7 mysqld 998 1 10.2 java -jar app.jar</pre>



3. MEMORY LEAKS

- A memory leak happens when a process keeps using more memory but never releases it.
- Over time, memory usage grows until the server runs out of RAM.
- Common in applications, poorly written code, or caching gone wrong.
- Watch for processes with constantly increasing MEM % or RES size.



LEAK TODAY, CRASH TOMORROW.



4. LINUX OOM KILLER

- When RAM + swap are full, Linux activates the OOM (Out Of Memory) killer.
- It kills a process to save the system.
- Check logs:

```
$ dmesg | grep -i oom
```
- Or check:

```
$ journalctl -k -g -i oom
```
- Killed processes disappear suddenly.
- Look for "Killed process" messages in logs.



5. FINDING MEMORY-HEAVY PROCESSES

- 1 Run: `top` and press M (sort by memory).
- 2 Identify the top consumers.
- 3 Check the command and why it uses so much memory.
- 4 Investigate: logs, configs, traffic, code, or data.



6. WHY ADDING MEMORY IS NOT ALWAYS THE REAL FIX

- More RAM only delays the problem.
- The leak or bad process will keep growing.
- You pay more but the root cause remains.
- Real fix = fix the application, configuration, or workload causing high memory usage.
- Monitor, find the cause, fix it permanently.



7. QUICK TROUBLESHOOTING FLOW



DON'T JUST RESTART. UNDERSTAND. FIX. PREVENT.

PRODUCTION DISK IS 100% FULL



A full disk can crash apps, stop services, and cause data loss.



1. WHY FULL DISKS BREAK APPLICATIONS

- No space to write logs, temp files, or data.
- Databases cannot write or commit.
- Applications fail to save sessions, uploads, or cache.
- Services may crash or become unresponsive.
- Even system processes can fail.
- Can lead to corruption and downtime.



2. df -h VS du -sh

COMMAND	WHAT IT SHOWS
df -h	Shows filesystem disk usage. Tells you which mount/partition is full. Does NOT show what is using the space.
du -sh	Shows disk usage of files and directories. Helps you find what is using the space.



3. COMMON SPACE CONSUMERS

	Logs	/var/log, application logs
	Temporary files	/tmp, /var/tmp
	Container data	/var/lib/docker, /var/lib/containerd
	Application artifacts	Builds, uploads, caches
	Databases	Data files, WAL, backups
	Mail spools	/var/spool/*



4. FINDING LARGE DIRECTORIES & FILES

Check filesystem usage	See which filesystem is full.
df -h	
Find large directories (top level)	Finds big directories in root.
du -xh --max-depth=1 /	
Find large directories (inside a path)	Check common paths like /var/log.
du -xh --max-depth=1 /var/log	
Find large files (anywhere)	Find files larger than 100MB.
find / -type f -size +100M -exec du -h {} +	
Find and sort all usage (largest first)	Shows top 20 biggest files and dirs.
du -ah / sort -rh head -n 20	



5. DELETED FILES STILL HELD OPEN

- A process may delete a file, but still keep the space open.
- **df -h** shows space is still used.
- **du** won't show the file (it's deleted).
- Find with:
ls -l | grep '(deleted)'
- Restart the service or process to release space.



6. SAFE CLEANUP VS DANGEROUS DELETION

SAFE CLEANUP ✓

- ✓ Delete old logs only.
- ✓ Rotate logs first.
- ✓ Remove temp files.
- ✓ Remove unused packages.
- ✓ Prune old Docker images, containers, volumes.
- ✓ Clean application caches if safe.

DANGEROUS DELETION ✗

- ✗ **rm -rf /** (Never!)
- ✗ Deleting files you don't understand.
- ✗ Removing DB files.
- ✗ Deleting active logs.
- ✗ Deleting mounts or system directories.



7. LOG ROTATION (PREVENTS DISK FULL)

- Log rotation keeps logs from growing forever.
- Compresses old logs and deletes very old ones.
- Use logrotate on Linux servers.
- Example config: /etc/logrotate.d/
- Always test your log rotation rules.



8. QUICK CHECKLIST

- 1 Check: **df -h**
- 2 Identify the full partition.
- 3 Use **du -sh** to find large directories.
- 4 Find large files and deleted open files.
- 5 Clean up safely or rotate logs.
- 6 Verify: **df -h** (space is free).



JUNIOR ENGINEER TIP

Monitoring disk usage and configuring log rotation prevents incidents before users feel the impact.

APPLICATION RETURNS HTTP 500 ERRORS



HTTP 500 = Internal Server Error. The server received the request but something went wrong while processing it.

1. WHAT HTTP 500 ACTUALLY TELLS YOU

- The request reached the server.
- The server could not complete the request due to an unexpected condition.
- It is a generic error—not the real cause.
- The real problem is in your application, configuration, dependencies, or data.

2. WHY THE BROWSER ERROR IS ONLY A SYMPTOM



- The browser only shows the final result.
- It hides useful details, stack traces, and logs.
- Many different problems can produce HTTP 500.
- Always check server-side logs for the real cause.
- Never rely on the browser error message.

3. CHECK APPLICATION LOGS FIRST

WHERE LOGS LIVE (EXAMPLES)	CHECK WITH
Linux servers /var/log/, /var/log/<app>/, /opt/<app>/logs/	cat, less, tail -f, grep -i error
Docker containers docker logs <container>	docker logs, --tail, -f
Kubernetes kubectl logs <pod> -n <namespace>	kubectl logs -f, --previous
Cloud / PaaS Use provider log services (CloudWatch, Log Explorer, etc.)	Use the provider console / CLI

4. COMMON CAUSES OF HTTP 500

CODE ERRORS	Bugs, exceptions, null pointer, unhandled errors.
CONFIGURATION FAILURES	Wrong config, missing environment variables, invalid values.
DEPENDENCY FAILURES	Downstream services, APIs, or internal services failing.
DATABASE FAILURES	Connection failures, timeouts, slow queries, locks.
EXTERNAL API FAILURES	Rate limits, timeouts, invalid responses, auth failures.
PERMISSION ISSUES	File access, service accounts, or missing secrets.

5. DIAGNOSIS CHECKLIST

- 1 Reproduce the error (if safe).
- 2 Check application logs around the error time.
- 3 Look for exceptions, stack traces, or error codes.
- 4 Check recent configuration changes.
- 5 Check dependency health (DB, APIs, services).
- 6 Check server resources (CPU, Memory, Disk).
- 7 Check database connectivity and slow queries.
- 8 Correlate the error with recent deployments.



6. CORRELATE ERRORS WITH RECENT DEPLOYMENTS

- 1 Check deployment history (who, what, when).
- 2 Review release notes and changes.
- 3 Look for new features or dependencies.
- 4 Compare error start time vs deployment time.
- 5 Rollback if the change is the likely cause.
- 6 Always verify after rollback or fix.



7. USEFUL COMMANDS

COMMAND	WHAT IT DOES
curl -I http://yourapp.com	Check if the server returns 500 quickly.
curl -v http://yourapp.com	See detailed response and connection info.
tail -f /var/log/<app>/error.log	Watch logs in real time.
grep -i error /var/log/<app>/*	Find error lines in logs.
systemctl status <app>	Check application service status.

8. WHAT TO DO (RULES)

- ✓ Do not panic.
- ✓ Do not restart blindly.
- ✓ Collect logs and facts first.
- ✓ Identify the root cause.
- ✓ Fix the cause, not just the symptom.
- ✓ Verify that the error is gone for real.



JUNIOR ENGINEER TIP

HTTP 500 is not the problem.
It is your clue to start real troubleshooting.

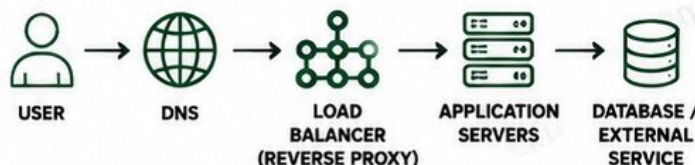
USERS RECEIVE 502, 503 AND 504 ERRORS

UNDERSTAND THE DIFFERENCE. FIND THE FAILING LAYER. FIX THE REAL PROBLEM.

1. DIFFERENCE BETWEEN 502, 503 AND 504

ERROR	WHAT IT MEANS	COMMON CAUSES	WHO RETURNS IT
502 Bad Gateway	The server (proxy/load balancer) received an invalid response from the upstream server.	- App crashed - Invalid response - Connection reset - Upstream closed unexpectedly	Proxy / Load Balancer
503 Service Unavailable	The server is available but cannot handle the request right now.	- Upstream down - Overloaded - Maintenance mode - No healthy servers	Proxy / Load Balancer
504 Gateway Timeout	The server (proxy/load balancer) connected to the upstream, but it did not respond in time.	- Upstream slow - Long-running request - App or DB timeout - Network latency	Proxy / Load Balancer

2. PROXY & LOAD-BALANCER REQUEST FLOW



- 1 User sends a request to your domain.
- 2 DNS resolves the domain to the load balancer IP.
- 3 Load balancer accepts the request.
- 4 It forwards the request to a healthy backend server.
- 5 The application processes the request.
- 6 The response goes back the same way to the user.



TIP: 502, 503, 504 usually mean the problem is between the Load Balancer and your backend.

3. BACKEND UNAVAILABLE vs UNHEALTHY vs TOO SLOW



UNAVAILABLE

The backend is down or not accepting connections.

Result:
Usually 503 or 502
Check:

- Server is running?
- Port open?
- Firewall / SG?



UNHEALTHY

Backend is running but failing health checks.

Result:
503
Check:

- Health check path
- Application errors
- Resource issues



TOO SLOW

Backend is running but taking too long to respond.

Result:
504
Check:

- Long DB queries
- External API slow
- High latency

4. WHAT LOAD BALANCER HEALTH CHECKS DO



- Regularly check if your backend is healthy.
- If a server fails the check, it is removed from the rotation.
- Prevents traffic from going to broken servers.
- Common checks: HTTP / HTTPS / TCP / gRPC
- Make sure the health check path is fast and reliable.

EXAMPLE (AWS ALB)

- Protocol: HTTP
- Path: /health
- Interval: 10s
- Timeout: 5s
- Healthy Threshold: 2
- Unhealthy Threshold: 2

Tune these values carefully. Too strict causes false alarms. Too loose causes slow failure detection.

5. APPLICATION PORTS

PORT	COMMON SERVICE	NOTES
80	HTTP	Web traffic (not secure)
443	HTTPS	Web traffic (secure)
22	SSH	Server access
3306	MySQL	Database connections
5432	PostgreSQL	Database connections
6379	Redis	Cache / Key-Value Store
8080	App (Tomcat, Node, etc.)	Common app port
3000	Node.js / React	Common dev port

6. CHECK UPSTREAM LOGS

LOAD BALANCER / PROXY LOGS

- Shows status codes returned by backend.
- Shows target IP, response time, errors.
- Essential to see what LB experienced.

APPLICATION LOGS

- Shows actual errors and stack traces.
- Explains why the app failed.

SERVER / SYSTEM LOGS

- Check OS logs, service logs, and metrics.
- Look for crashes, OOM, high CPU, etc.

7. DIAGNOSING THE FAILING LAYER

1	CAN USERS REACH THE LOAD BALANCER?	curl -I https://your-domain.com
2	IS THE LOAD BALANCER HEALTHY?	Check LB metrics and dashboard.
3	ARE BACKENDS HEALTHY?	Check health check status / target group.
4	IS THE APPLICATION RESPONDING?	curl http://backend-ip:port or check app logs.
5	IS THE DATABASE / DEPENDENCY RESPONDING?	Test DB / external service connectivity.
6	CHECK LOGS AND METRICS AT EACH LAYER.	Follow the flow. Find the break.



REMEMBER

IDENTIFY THE FAILING LAYER FIRST. THEN APPLY THE RIGHT FIX.



JUNIOR ENGINEER TIP

Don't guess. Collect evidence. Follow the flow. Solve the right problem.



veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

DNS RESOLUTION FAILURE

APPLICATION WORKS BY IP BUT NOT HOSTNAME

1. WHAT IS HAPPENING?

- You can reach the application using the IP address, but the hostname does not resolve to the correct IP.
- This is a DNS resolution problem.
- Caused by wrong records, resolver issues, caching, or propagation delays.

2. COMMON SIGNS

- ✗ Site not loading / DNS_PROBE_FINISHED_NXDOMAIN
- 🕒 Intermittent DNS failures
- ! Hostname resolves to wrong IP
- ? Works on one network but not another

3. ESSENTIAL DNS TOOLS

TOOL	WHAT IT DOES	EXAMPLE
dig	Detailed DNS lookup. Shows records, TTL, and authoritative answers.	<code>dig example.com</code> <code>dig example.com A</code> <code>dig example.com MX</code>
nslookup	Simple DNS lookup. Good for quick checks.	<code>nslookup example.com</code> <code>nslookup -type=mx example.com</code>
getent hosts	Shows how the system resolves a hostname (using /etc/resolv.conf and NSS).	<code>getent hosts example.com</code>
host	Another simple DNS lookup tool.	<code>host example.com</code>

4. DNS RECORDS YOU SHOULD KNOW

RECORD	PURPOSE	EXAMPLE
A	Maps hostname to IPv4 address.	<code>example.com. 3600 IN A 203.0.113.10</code>
AAAA	Maps hostname to IPv6 address.	<code>example.com. 3600 IN AAAA 2001:db8::10</code>
CNAME	Alias from one name to another name.	<code>www.example.com. 3600 IN CNAME example.com.</code>
MX	Mail exchange mail routing.	<code>example.com. 3600 IN MX 10 mail.example.com.</code>
NS	Authoritative name servers for the domain.	<code>example.com. 86400 IN NS ns1.example.com.</code>
TXT	Text information (SPF, verification, etc).	<code>example.com. 3600 IN TXT "v=spf1 include:_spf.example.com ~all"</code>

5. CHECK DNS RESOLUTION (EXAMPLES)

- dig**
- ```
dig example.com +short
;; Should return the IP address
dig example.com A
;; Check A record
dig example.com MX
;; Check mail servers
```
- nslookup**
- ```
nslookup example.com
;; Should return Server and Address
nslookup -type=a example.com
;; Check A record
```
- getent hosts**
- ```
getent hosts example.com
;; Shows what your system resolves
```

## 6. COMMON CAUSES

- 📄 Wrong DNS records (incorrect IP, CNAME, MX, etc).
- 🗄️ Missing DNS records.
- 🕒 DNS propagation delay (changes not visible everywhere yet).
- 🖥️ Caching at resolver, ISP, browser, or OS.
- 🌐 Incorrect nameservers or delegation.
- ⚙️ Resolver misconfiguration on the server.
- 🔥 Firewall blocking DNS (UDP/TCP port 53).

## 7. TTL AND CACHING EXPLAINED

- 🕒 **TTL (Time To Live)** is the time (in seconds) a DNS record can be cached by resolvers.
- 📄 **High TTL** = Changes take longer to show everywhere.
- 🔱 **Low TTL** = Changes propagate faster.
- 🔄 Even after you change a DNS record, some users may still see the old record until cache expires.
- 👥 **Why changing DNS can affect production gradually:** Different ISPs, resolvers, and devices have different caches.

## 8. WHY CHANGING DNS AFFECTS PRODUCTION GRADUALLY



This process can take seconds to 48+ hours depending on TTL.

## 9. QUICK CHECKLIST WHEN DNS FAILS

- ✓ Can users reach the application by IP?
- ✓ Does the hostname resolve to the correct IP?
- ✓ Are DNS records correct and complete?
- ✓ Check TTL and propagation status.
- ✓ Test from multiple networks and resolvers.
- ✓ Clear local cache only for testing (do not rely on it).



### JUNIOR ENGINEER TIP

Always test DNS from the server and from outside the network. Problems can be location-specific!



# CONNECTION REFUSED VS CONNECTION TIMEOUT

WHY THESE ERRORS MEAN DIFFERENT THINGS

## 1. CONNECTION REFUSED

The server is reachable, but nothing is listening on that port.



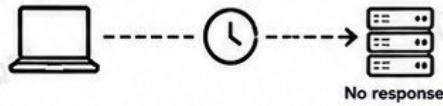
- The network path is working. **Error: Connection refused**
- The target host is alive.
- The port is closed or no service is listening.
- You get an immediate rejection (RST).

### Common causes:

- Wrong port
- Service is not running
- Application crashed
- Listening on the wrong interface (localhost only)
- Firewall with REJECT rule

## 2. CONNECTION TIMEOUT

The server is unreachable, or packets are being dropped.



### Error: Connection timed out

- No response is received within the timeout period.
- Packets are dropped somewhere in the path.
- You get no reply (no RST, no response).

### Common causes:

- Host is down
- Firewall / security group blocking
- Network routing issue
- Wrong IP address
- Packet dropped by ACL / NACL / Firewall

## QUICK DIFFERENCE

### REFUSED

= We can reach the host, but the port is not open.



### TIMEOUT

= We cannot reach the host or packets are blocked.



### TIP

Refused is fast.  
Timeout is slow  
(waits until it gives up).

## 3. CHECK LISTENING SERVICES

Check what ports and services are listening on the server.

```
ss -lntp
```

| FLAG | WHAT IT SHOWS               | EXAMPLE  |
|------|-----------------------------|----------|
| -l   | Listening sockets only      | ss -lntp |
| -n   | Show numeric addresses      | ss -lntp |
| -t   | TCP sockets                 | ss -lntp |
| -u   | UDP sockets                 | ss -lnup |
| -p   | Show the process (PID/Name) | ss -lntp |

Example output:

| State  | Recv-Q | Send-Q | Local Address:Port | Peer Address:Port | Process                |
|--------|--------|--------|--------------------|-------------------|------------------------|
| LISTEN | 0      | 128    | 0.0.0.0:80         | 0.0.0.0:*         | users:(nginx,pid=1234) |
| LISTEN | 0      | 128    | 0.0.0.0:22         | 0.0.0.0:*         | users:(sshd,pid=789)   |
| LISTEN | 0      | 100    | 127.0.0.1:3306     | 0.0.0.0:*         | users:(mysqld,pid=456) |



If the port is not listed here, nothing is listening on it.

## 4. FIREWALLS & SECURITY GROUPS

Traffic can be blocked on the host or in the cloud.

On Linux (local firewall):

```
sudo ufw status verbose
sudo iptables -L -n -v
sudo firewall-cmd --list-all
```

In the Cloud (examples):

- AWS Security Group
- Azure NSG
- GCP Firewall Rules



Check both inbound and outbound rules.  
Even outbound blocks can cause timeouts.

## 5. ROUTING & NETWORK PATHS

If the path to the host is broken or unreachable, you will get a timeout.

Useful commands:

```
ping <host or ip>
traceroute <host or ip>
mtr <host or ip>
```

What to look for:

- No response at all
- Hops timing out
- Wrong route
- ISP / Network issue
- Private network not reachable
- Cloud VPC peering / route table issue



If you can't reach the host (ping/traceroute fails), expect a TIMEOUT.

## 6. WRONG PORTS

Many "connection refused" errors are simply caused by using the wrong port.

Common mistakes:

- App runs on 8080, you try 80
- DB runs on 5432, you try 3306
- Service moved but config not updated
- Typo in port number

Test quickly:

```
curl -v http://<host>:<port>
nc -vz <host> <port>
```



Always confirm the correct port from documentation or config.

## 7. BUILDING THE CORRECT TROUBLESHOOTING MENTAL MODEL



### 1. OBSERVE

What error do you see?  
Refused or Timeout?



### 2. TEST REACHABILITY

Can you reach the host?  
ping / traceroute



### 3. CHECK NETWORK PATH

Is the network path working?  
Look for drops.



### 4. CHECK FIREWALLS

Could a firewall be blocking the traffic?



### 5. CHECK SERVICE

Is the service listening?  
ss -lntp



### 6. VERIFY PORT

Are you using the correct port?



### 7. VERIFY & FIX

Apply the fix and test again.



Always gather evidence before making changes. Change one thing at a time. Then verify.

## 8. QUICK CHECKLIST

- ✓ Is the error REFUSED or TIMEOUT?
- ✓ Can I reach the host (ping/traceroute)?
- ✓ Is the port open and service listening (ss -lntp)?
- ✓ Is the firewall / security group allowing traffic?
- ✓ Am I using the correct IP and port?
- ✓ Check logs if the service is running but still failing.

## JUNIOR ENGINEER TIP



Most incidents are not random.  
They are the result of something specific.  
Follow a process. Don't guess.  
Good engineers troubleshoot systematically.



veriqta



veriqta



veriqta



veriqta



veriqta



veriqta



# SSH ACCESS TO A PRODUCTION SERVER FAILS

DIAGNOSE THE REAL CAUSE. FIX THE RIGHT LAYER.

## 1. TIMEOUT VS PERMISSION DENIED

### TIMEOUT



No response from the server.

The connection could not be established.

#### Common causes:

- Network unreachable
- Security group / firewall
- Wrong IP or port
- SSH service down

### PERMISSION DENIED



Connection reached the server, but authentication failed.

#### Common causes:

- Wrong username
- Wrong / missing key
- Bad key permissions
- Not authorized on server

## 2. SSH KEYS & PERMISSIONS



Private key file permissions **MUST** be 400 or 600.

```
chmod 600 ~/.ssh/id_rsa
chmod 400 ~/.ssh/id_rsa.pub
```

#### Why?

- If permissions are too open, SSH will ignore the key.
- Open permissions are a security risk.



**TIP:** Never use chmod 777 on SSH keys or .ssh folder.

## 3. USERNAME MISTAKES



- Using the wrong username is very common.
- Many servers use a non-root user (ec2-user, ubuntu, admin, deploy, etc).

#### Try the correct user:

```
ssh -i key.pem ec2-user@<ip>
```



**TIP:** Don't guess. Check the server docs or ask the team.

## 4. NETWORK ACCESS CHECKLIST



Ensure you can reach the server on port 22. Use these commands:

```
ping <server-ip>
nc -vz <server-ip> 22
telnet <server-ip> 22
```

#### If these fail, check:

- Internet connectivity
- Security group / firewall
- NACL / network ACL
- Route table / routing
- VPN / Bastion / Proxy

## 5. SECURITY GROUPS / FIREWALLS



Ensure inbound rule allows TCP port 22 from your IP.

#### Example (AWS Security Group):

| Type | Protocol | Port | Source              |
|------|----------|------|---------------------|
| SSH  | TCP      | 22   | Your IP / 0.0.0.0/0 |

#### On the server (Linux firewall):

```
sudo ufw status
sudo firewall-cmd --list-all
sudo iptables -L -n -v
```

## 6. USE SSH VERBOSE MODE



Verbose mode shows what SSH is doing. Use it to find the exact failure point.

```
ssh -v -i key.pem user@<ip>
```

#### Look for messages like:

- Connecting to ...
- Connection established
- Offering public key
- Permission denied
- Connection timed out

## 7. CHECK SSH SERVICE STATE



SSH service must be running on the server.

#### On systemd systems:

```
sudo systemctl status sshd
sudo systemctl restart sshd
sudo systemctl enable sshd
```

#### On older systems:

```
sudo service ssh status
sudo service ssh restart
```

## 8. COMMON SSH ERRORS & MEANING

| ERROR MESSAGE                 | WHAT IT MEANS                                                   |
|-------------------------------|-----------------------------------------------------------------|
| Connection timed out          | Cannot reach the server on port 22.                             |
| Connection refused            | Server reachable but SSH not listening or blocked.              |
| Permission denied (publickey) | Authentication failed. Check user, keys, and permissions.       |
| No such file or directory     | Key file path is wrong.                                         |
| Host key verification failed  | Host key changed or first-time connection. Verify and continue. |

## 9. WHY CHMOD 777 IS NOT A FIX



- It makes your private key readable by everyone.
- SSH ignores keys with open permissions.
- It breaks security best practices.
- It can get your server access revoked.



**Use the correct permission:**  
`chmod 600 ~/.ssh/id_rsa`

## 10. JUNIOR ENGINEER TROUBLESHOOTING FLOW



## REMEMBER



SSH is a process, not magic. Find the failing layer. Fix the cause. Verify the result.



veriqta



veriqta



veriqta



veriqta



veriqta



veriqta



# A DEPLOYMENT BREAKS PRODUCTION

EVERYTHING WORKED BEFORE DEPLOYMENT.

FIND WHAT CHANGED. RESTORE SERVICE. THEN FIX THE REAL CAUSE.

## 1. START WITH IMPACT



- What is broken?
- Who is affected?
- When did it start?
- What is the business impact?
- Is it getting better or worse?



**TIP:** Understand the impact before you change anything.

## 2. ESTABLISH A TIMELINE



- When was the last known good state?
- When was the deployment executed?
- What happened right after the deployment?
- Any alerts, spikes, or errors at that time?



A clear timeline narrows down the search.

## 3. CHECK WHAT CHANGED



- New code version?
- New configuration?
- New environment variables?
- New dependencies or library versions?
- Infrastructure changes (ports, resources, scaling)?
- Database schema changes?



Changes introduce risk. Compare before vs after.

## 4. CHECK LOGS FOR EVIDENCE



### Application Logs

- Look for errors that started after deployment.
- Check stack traces and failure patterns.

### Deployment / Orchestration Logs

- CI/CD pipeline logs
- Container / Pod events
- Deployment rollout events
- Load balancer / Ingress logs



Logs tell you what happened. Read them before guessing.

## 5. COMMON REASONS DEPLOYMENTS BREAK



### CODE CHANGES

- Bugs in new features
- Unhandled edge cases
- Performance regressions



### CONFIGURATION CHANGES

- Wrong values
- Missing variables
- Misconfigured services



### DEPENDENCY CHANGES

- New version with breaking changes
- Removed APIs
- Incompatible libraries



### INFRASTRUCTURE CHANGES

- Wrong port / target port
- Insufficient resources
- Incorrect scaling rules



### EXTERNAL DEPENDENCIES

- Third-party API changes
- Database connectivity
- Rate limiting

## 6. ROLLBACK VS DEBUGGING IN PLACE

### ROLLBACK WHEN:

- ✓ Users are heavily impacted
- ✓ You don't know the root cause
- ✓ The change is high risk
- ✓ Quick recovery is the priority
- ✓ You can rollback safely



Rollback restores the last known good state.

### DEBUG IN PLACE WHEN:

- ✓ Low user impact
- ✓ Easy to test a hypothesis
- ✓ Rollback is risky or complex
- ✓ You need more information to understand the issue



Debugging helps find the real root cause.

## 7. HOW TO ROLLBACK SAFELY

- 1 Identify the last known good version.
- 2 Confirm rollback plan and dependencies.
- 3 Execute rollback using your deployment tool.

```
Kubernetes
kubectl rollout undo deployment/my-app

Docker Swarm
docker service update --rollback my-service

Helm
helm rollback my-release <revision>
```



Rollback is not failure. It is incident response.

## 8. VERIFY SERVICE HEALTH AFTER ROLLBACK



### Check health endpoints

```
curl -f https://your-domain/health
```



### Check metrics and dashboards

Look for error rate, latency, traffic



### Check logs for errors

Ensure no new errors appear



### Verify dependencies are healthy

DB, cache, queue, external APIs



### Confirm users can access

Test critical user journeys



**JUNIOR ENGINEER TIP:** Always document what changed, what you observed, what you did, and the result. Today's incident is tomorrow's knowledge.



veriqta



veriqta



veriqta



veriqta



veriqta



veriqta



# DOCKER CONTAINER KEEPS EXITING

FIND THE CAUSE. FIX THE CAUSE. VERIFY THE FIX.



## RUNNING CONTAINER VS STOPPED CONTAINER



### RUNNING

Container is up, process inside is alive, and it is accepting work.



### STOPPED

Container has exited. The main process ended or crashed.



## SEE ALL CONTAINERS

```
docker ps -a
```

Shows running and stopped containers.

### KEY COLUMNS

|              |                     |
|--------------|---------------------|
| CONTAINER ID | Unique container ID |
| IMAGE        | Image used          |
| COMMAND      | Command that ran    |
| STATUS       | Up or Exited (code) |
| PORTS        | Published ports     |
| NAMES        | Container name      |



## SEE RECENT LOGS

```
docker logs <container_id | name>
```

Shows the output from the container before it exited.

### USEFUL FLAGS

|                                                |                          |
|------------------------------------------------|--------------------------|
| <code>docker logs -f &lt;id&gt;</code>         | Follow logs in real time |
| <code>docker logs --tail 100 &lt;id&gt;</code> | Last 100 lines           |
| <code>docker logs --since 5m &lt;id&gt;</code> | Last 5 minutes           |



## DEEP INSPECTION

```
docker inspect <container_id | name>
```

Shows low-level details. Very useful!

### CHECK THESE FIELDS

|                  |                            |
|------------------|----------------------------|
| State.Status     | running or exited          |
| State.ExitCode   | Exit code (why it stopped) |
| State.Error      | Error message if any       |
| State.StartedAt  | When it started            |
| State.FinishedAt | When it stopped            |
| Config.Cmd       | Command the container ran  |
| Config.Env       | Environment variables      |
| Mounts           | Volumes                    |
| NetworkSettings  | Networking details         |



## EXIT CODES (COMMON)

| EXIT CODE | MEANING                                     |
|-----------|---------------------------------------------|
| 0         | Success (normal exit)                       |
| 1         | General error                               |
| 125       | Docker error (container could not start)    |
| 126       | Command found but not executable            |
| 127       | Command not found                           |
| 137       | Killed (SIGKILL) - often OOM or manual kill |
| 139       | Segmentation fault                          |
| 143       | Stopped gracefully (SIGTERM)                |



## COMMON REASONS CONTAINERS EXIT



### BAD COMMAND

Wrong command or typo. Container exits immediately.



### MISSING ENVIRONMENT VARIABLES

Application needs a value that is not set.



### APPLICATION CRASHES

Bug in code, bad configuration, or unexpected input.



### MISSING DEPENDENCIES

Database, service, or file the app needs is not available.



### PERMISSION PROBLEMS

Application cannot read/write required files or directories.



## WHY REPEATEDLY RUNNING `docker restart` IS NOT DIAGNOSIS

- It hides the real problem instead of fixing it.
- It wastes time and increases downtime.
- The container will exit again if the cause is still there.
- Engineers solve problems, not symptoms. Find the cause once and fix it.
- You lose the logs and context.



## JUNIOR ENGINEER TIP

Always gather evidence first:

- `docker ps -a`
  - `docker logs <id>`
  - `docker inspect <id>`
- Then make a decision.



## TROUBLESHOOTING WORKFLOW



VERIQTA | LEARN. PRACTICE. DEPLOY. REPEAT.



veriqta



veriqta



veriqta



veriqta



veriqta



veriqta



# DOCKER APPLICATION CANNOT BE REACHED

CONTAINER IS RUNNING BUT USERS CANNOT CONNECT

## WHAT'S HAPPENING?

The container is running, but the application inside is not reachable from your local machine or other users.

This is almost always a networking or port issue.

## CONTAINER PORT VS HOST PORT

| TYPE           | WHERE IT LIVES                                          | EXAMPLE          |
|----------------|---------------------------------------------------------|------------------|
| CONTAINER PORT | Inside the container. Application listens on this port. | 80, 8080, 3000   |
| HOST PORT      | On your machine. Traffic from outside comes here.       | 8080, 3000, 5000 |



### RULE:

Traffic flows from your machine's host port to the container port.

## PORT PUBLISHING

Use `-p` or `-P` to publish a container port to the host.

### EXAMPLE

```
docker run -d -p 8080:80 nginx
```

↑  
HOST PORT  
(Your machine)

↓  
CONTAINER PORT  
(Inside container)

• Now you can access:

`http://localhost:8080`

## APPLICATION BINDING MATTERS

If the application inside the container is listening only on localhost (127.0.0.1), it will NOT be reachable from outside.



### BAD (NOT REACHABLE)

```
app.listen(127.0.0.1, 3000)
```

Only accepts connections from inside the container.

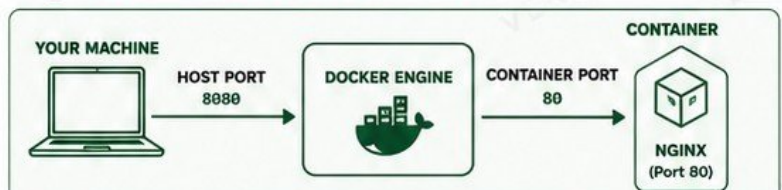


### GOOD (REACHABLE)

```
app.listen(0.0.0.0, 3000)
```

Accepts connections from anywhere (outside too).

## DOCKER NETWORKING BASICS



- Traffic comes to HOST PORT on your machine.
- Docker forwards it to CONTAINER PORT.
- The app inside must be listening on 0.0.0.0.

## COMMON CHECKS & COMMANDS

### 1. CHECK RUNNING CONTAINERS

```
docker ps
```

- See container status
- See PORTS column
- Example: `0.0.0.0:8080->80/tcp`

### 2. INSPECT PORT DETAILS

```
docker inspect <container_id>
```

- Check "NetworkSettings"
- Look for "Ports" mapping
- Confirms host port → container port

### 3. TEST CONNECTIVITY

```
curl -v http://localhost:8080
```

- `-v` shows connection details
- Connection refused? Port not open
- Timeout? Firewall or wrong port
- Works locally but not remotely? Check firewall / security group

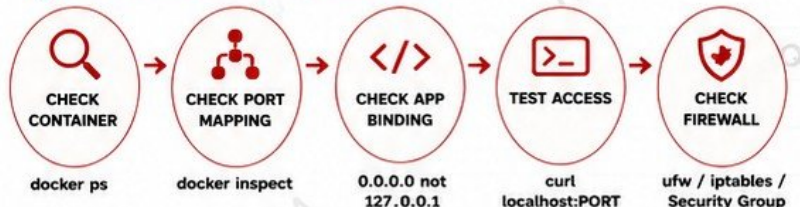
## HOST FIREWALL CONSIDERATIONS

Even if Docker is configured correctly, the host firewall can block access.

- Linux (UFW)  
`sudo ufw allow 8080/tcp`
- Linux (iptables)  
`sudo iptables -A INPUT -p tcp --dport 8080 -j ACCEPT`
- Cloud Security Groups  
Allow inbound traffic on the host port.



## QUICK TROUBLESHOOTING FLOW



### JUNIOR ENGINEER TIP

Always confirm each layer:  
Container → Port Mapping → App Binding → Firewall.  
Don't guess. Test.



### REMEMBER THIS

If a container is running but unreachable, it is almost always a networking or port configuration problem.



veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

VERIQTA | LEARN. PRACTICE. DEPLOY. REPEAT.



# KUBERNETES POD IS IN CRASHLOOPBACKOFF

FIND THE CAUSE. FIX THE CAUSE. DON'T JUST DELETE THE POD.

## ? WHAT IS CRASHLOOPBACKOFF?

CrashLoopBackOff means the container starts, crashes, and Kubernetes keeps restarting it.

Kubernetes is backing off between restarts to avoid overwhelming the node.

The Pod is not ready and your application is unavailable.



## > 1. CHECK POD STATUS

```
kubectl get pods -A
```

Look for STATUS column.

| STATUS           | MEANING                           |
|------------------|-----------------------------------|
| Running          | Pod is running fine.              |
| CrashLoopBackOff | Container is crashing repeatedly. |
| ImagePullBackOff | Cannot pull container image.      |
| Pending          | Pod is not scheduled yet.         |

## 📄 2. DESCRIBE THE POD

```
kubectl describe pod <pod-name> -n <namespace>
```

Check these important sections:

- Events (at the bottom)
- State: Waiting (Reason)
- Last State
- Restart Count
- Readiness/Liveness Probe events
- Environment variables
- Mounted volumes
- Image and command



## 📄 3. CHECK POD LOGS

```
kubectl logs <pod-name> -n <namespace>
```

Shows logs from the current container instance.



### TIP

Logs tell you why the app is crashing.



## 🕒 4. CHECK PREVIOUS LOGS

```
kubectl logs <pod-name> -n <namespace> --previous
```

Shows logs from the last crashed container instance.



### TIP

This is critical for containers that crash on startup.

## ⚠️ COMMON CAUSES OF CRASHLOOPBACKOFF



### BAD CONFIGURATION

- Wrong config file
- Invalid YAML
- Wrong arguments
- Missing required settings



### MISSING SECRETS / CONFIGMAPS

- Secret not found
- Wrong key name
- No permissions
- Mount failures



### APPLICATION CRASHES

- Code bug
- Null pointer exception
- Segmentation fault
- Out of memory



### PROBE FAILURES

- Liveness probe failing
- Readiness probe failing
- Startup probe failing
- Wrong probe path/port



### DEPENDENCY ISSUES

- Database not reachable
- External API down
- DNS resolution failure
- Connection refused

## 🛡️ STOP DELETING PODS. FIX THE CAUSE.



### DON'T DO THIS

- Deleting the Pod
- `kubectl delete pod <pod-name>`
- It will be recreated and crash again.
- This does NOT fix the problem.



### DO THIS INSTEAD

- Find the root cause.
- Fix the configuration, code, secret, or dependency.
- Then let Kubernetes restart the Pod.
- Or redeploy the workload.



## POD RESTART BEHAVIOR

Kubernetes uses exponential backoff:



This continues until the issue is fixed.

## ⚙️ TROUBLESHOOTING WORKFLOW



### REMEMBER

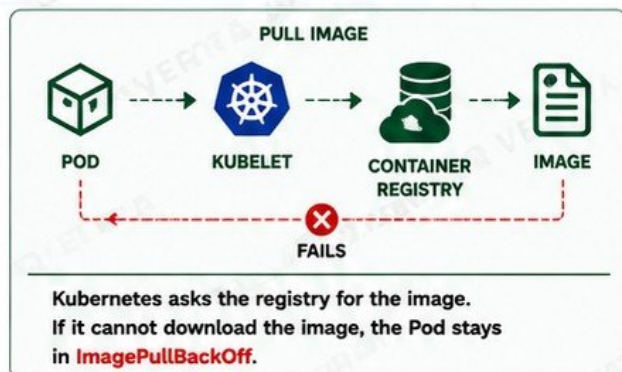
KUBERNETES IS TELLING YOU SOMETHING. LISTEN TO THE LOGS. FIX THE CAUSE. THEN VERIFY.



# KUBERNETES POD SHOWS IMAGEPULLBACKOFF

KUBERNETES CANNOT PULL THE CONTAINER IMAGE.

## WHAT KUBERNETES IS TRYING TO DO



## COMMON REASONS

|  |                                |                                                                        |
|--|--------------------------------|------------------------------------------------------------------------|
|  | <b>WRONG IMAGE / TAG</b>       | Image name or tag is incorrect or does not exist.                      |
|  | <b>MISSING IMAGE</b>           | Image has not been pushed to the registry.                             |
|  | <b>REGISTRY AUTHENTICATION</b> | Private registry requires credentials but they are missing or invalid. |
|  | <b>NETWORK ACCESS</b>          | Cluster cannot reach the registry due to network, DNS or firewall.     |
|  | <b>REGISTRY REJECTED</b>       | Rate limit, permission denied, or repository not accessible.           |

## 1. CHECK THE POD STATUS

```
kubectl get pods -A
```

Look for STATUS column.

| STATUS                  | MEANING                     |
|-------------------------|-----------------------------|
| <b>ImagePullBackOff</b> | Back-off pulling the image. |
| <b>ErrImagePull</b>     | Failed to pull the image.   |
| <b>Pending</b>          | Image pull in progress.     |

## 2. DESCRIBE THE POD

```
kubectl describe pod <pod-name> -n <namespace>
```

Check these important fields in the output:

- Events (at the bottom)
- Image
- Reason (ErrImagePull / ImagePullBackOff)
- Message (tells you why it failed)
- Back-off pulling image
- Image ID (if any)

## 3. EXAMINE EVENTS (FROM DESCRIBE)

| REASON                  | WHAT IT MEANS                           |
|-------------------------|-----------------------------------------|
| <b>ErrImagePull</b>     | Initial attempt to pull image failed.   |
| <b>ImagePullBackOff</b> | Kubernetes is backing off and retrying. |
| <b>Failed</b>           | Image pull failed.                      |
| <b>Unauthorized</b>     | Authentication failed.                  |
| <b>NotFound</b>         | Image or repository not found.          |
| <b>TooManyRequests</b>  | Rate limit reached.                     |
| <b>NetworkError</b>     | Cannot reach registry.                  |

## 4. CHECK POD LOGS (IF AVAILABLE)

```
kubectl logs <pod-name> -n <namespace>
```

Often empty for image pull errors.

Events from **describe pod** are more useful here.



### TIP

Image pull errors occur before the container starts, so application logs may not exist.

## 5. IMAGE PULL SECRETS

For private registries, create and use pull secrets.

```
Create secret (example for Docker Hub / OCI)
kubectl create secret docker-registry regcred \
 --docker-server=registry.example.com \
 --docker-username=USERNAME \
 --docker-password=PASSWORD \
 --docker-email=EMAIL -n <namespace>
```

Use in Pod or ServiceAccount:

- In Pod spec:

```
imagePullSecrets:
- name: regcred
```
- On ServiceAccount:

```
kubectl patch serviceaccount default -n <ns> \
-p '{"imagePullSecrets":[{"name":"regcred"}]}'
```

## 6. VERIFY THE EXACT IMAGE REFERENCE

Check the image in your deployment or pod template.

```
kubectl get deploy <deployment-name> -n <namespace> -o yaml
```

Look for:

```
spec:
 template:
 spec:
 containers:
 - name: <container-name>
 image: registry.example.com/namespace/app:tag
```

Make sure:

- Registry URL is correct
- Repository path is correct
- Image name spelling is correct
- Tag exists (use a real tag, not latest in production)
- Image has been pushed to the registry

## 7. NETWORK ACCESS CHECKS

From a node or debug pod:

```
DNS resolution
dig registry.example.com

TCP connectivity
telnet registry.example.com 443
or
nc -zv registry.example.com 443

Test pull manually (if Docker is available)
docker pull registry.example.com/namespace/app:tag
```

Also check:

- Firewall rules
- Proxy settings
- Egress restrictions (NetworkPolicy, Cloud SG)
- DNS configuration

## STOP DELETING PODS. FIX THE CAUSE.

### DON'T DO THIS

- Don't delete the Pod repeatedly.
- Don't restart the Deployment blindly.
- Don't scale up.
- Don't wait and hope it fixes itself.

### DO THIS INSTEAD

- Read the events and message carefully.
- Fix the image name, tag, auth or network issue.
- Verify the image exists and is accessible.
- Then let Kubernetes create a new Pod.

## QUICK FLOW



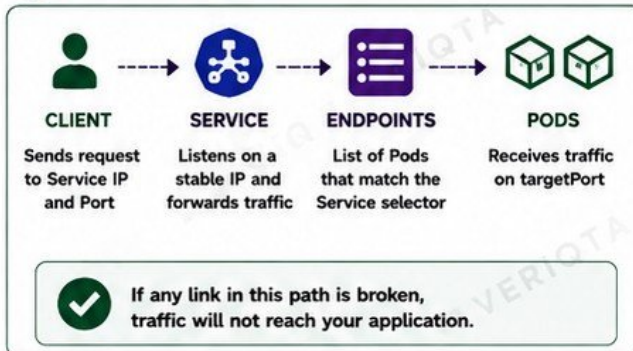
**REMEMBER:** IMAGEPULLBACKOFF IS A SYMPTOM. THE REAL PROBLEM IS WHY THE IMAGE CANNOT BE PULLED.



# KUBERNETES SERVICE HAS NO TRAFFIC

PODS ARE HEALTHY BUT APPLICATION IS UNREACHABLE

## 1 THE TRAFFIC PATH



## 2 COMMON REASONS

|  |                                  |                                                               |
|--|----------------------------------|---------------------------------------------------------------|
|  | <b>LABEL / SELECTOR MISMATCH</b> | Service selector does not match Pod labels.                   |
|  | <b>NO ENDPOINTS</b>              | No Pods match the selector, so Endpoints list is empty.       |
|  | <b>WRONG PORTS</b>               | Service port or targetPort does not match the container port. |
|  | <b>POD NOT READY</b>             | Pod is not Ready, so it is not added to Endpoints.            |
|  | <b>NETWORK POLICIES</b>          | NetworkPolicy blocks traffic to or from the Pods.             |

## 3 CHECK SERVICES

```
kubectl get svc -A
```

Look for the Service IP, PORT(S), and TARGET-PORT.

| COLUMN     | MEANING                                 |
|------------|-----------------------------------------|
| NAME       | Service name                            |
| TYPE       | ClusterIP, NodePort, LoadBalancer, etc. |
| CLUSTER-IP | Virtual IP of the Service               |
| PORT(S)    | Port exposed by the Service             |
| AGE        | How long the Service has existed        |

## 4 CHECK ENDPOINTS

```
kubectl get endpoints <svc-name> -n <ns>
```

Shows the list of Pods behind the Service. If the ENDPOINTS column is <none>, your Service has no backends.

| ENDPOINTS COLUMN  | MEANING                                       |
|-------------------|-----------------------------------------------|
| IP:PORT           | Pods are selected and ready.                  |
| <none>            | No Pods match the selector or none are ready. |
| IP:PORT (partial) | Some Pods are ready, others are not.          |



### TIP

No Endpoints = No traffic to Pods.  
Fix the selector or Pod readiness.

## 5 LABELS AND SELECTORS

### POD LABEL EXAMPLE

```
kubectl get pods --show-labels
```

| Key  | Value    |
|------|----------|
| app  | web      |
| tier | frontend |

### SERVICE SELECTOR EXAMPLE (YAML)

```
spec:
 selector:
 app: web
 tier: frontend
```



### TIP

Selector must match Pod labels exactly for Endpoints to be created.

## 6 SERVICE PORTS VS TARGET PORTS

### SERVICE YAML EXAMPLE

```
spec:
 ports:
 - name: http
 port: 80 # Service port
 targetPort: 8080 # Container port
 selector:
 app: web
```

### PORT (Service Port)

Port exposed by the Service. Clients use this port.

### TARGETPORT (Container Port)

Port on the Pod where the application is actually listening.



### TIP

If the container listens on 8080 but Service targetPort is wrong, traffic will not reach the application.

## 7 TEST CONNECTIVITY FROM INSIDE THE CLUSTER

### RUN A TEMPORARY TEST POD

```
kubectl run netshoot --rm -it --image=nicolaka/netshoot -- /bin/bash
```

### TEST THE SERVICE

- nslookup <svc-name> # Resolve Service DNS
- curl -v http://<svc-name>:<port>
- wget -qO- http://<svc-name>:<port>
- nc -zv <svc-name> <port>



If you can reach the Service from inside the cluster but not from outside, the issue is likely with Ingress, Load Balancer, or Network.

## 8 USEFUL COMMANDS SUMMARY

|                                   |                                     |
|-----------------------------------|-------------------------------------|
| kubectl get svc -A                | List all Services in all namespaces |
| kubectl get endpoints -A          | List all Endpoints                  |
| kubectl get pods -o wide          | Check Pod IPs and status            |
| kubectl get pods --show-labels    | See Pod labels                      |
| kubectl describe svc <name>       | Details about a Service             |
| kubectl describe endpoints <name> | Details about Endpoints             |
| kubectl describe pod <name>       | Pod health and events               |

## 9 TROUBLESHOOTING CHECKLIST

- 1 Are Pods running and Ready? `kubectl get pods -o wide`
- 2 Do Pod labels match Service selector? `kubectl get pods --show-labels`
- 3 Does the Service have Endpoints? `kubectl get endpoints <svc-name>`
- 4 Are Service port and targetPort correct? `kubectl describe svc <name>`
- 5 Is the application listening on that port? `kubectl exec -it <pod> -- ss -lntp`
- 6 Can you reach the Service from inside the cluster? `curl http://<svc-name>:<port>`
- 7 Are NetworkPolicies or firewalls blocking traffic?



### REMEMBER:

A SERVICE IS ONLY A WAY TO REACH PODS. IF ENDPOINTS ARE EMPTY, OR THE PODS ARE UNREACHABLE, THE SERVICE HAS NOWHERE TO SEND TRAFFIC.



veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

VERIQTA | LEARN. PRACTICE. DEPLOY. REPEAT.



# DATABASE CONNECTIONS ARE EXHAUSTED



## APPLICATION BECOMES SLOW OR UNAVAILABLE

### CONNECTION POOLS



Applications use a pool of connections instead of opening a new connection for every request.

### TOO MANY APPLICATION CONNECTIONS



More connections than the database can handle leads to slow performance or failures.

### LONG-RUNNING QUERIES



Expensive or unindexed queries hold connections for too long and block other work.

### CONNECTION LEAKS



Connections opened by the application are not closed, slowly exhausting the available pool.

### DATABASE LIMITS



Every database has a maximum number of connections. Hitting the limit breaks apps.



## WHY BLINDLY INCREASING THE CONNECTION LIMIT CAN MAKE INCIDENTS WORSE

- More connections consume more memory on the database.
- It increases CPU context switching.
- It can starve the server of resources.
- It hides the real problem (leaks, bad queries, or poor design).
- Eventually the database becomes unstable or crashes.

**FIX THE ROOT CAUSE, NOT JUST THE SYMPTOM.**



### KEY DIAGNOSTIC COMMANDS

| TOOL / COMMAND                             | WHAT IT SHOWS                               |
|--------------------------------------------|---------------------------------------------|
| SHOW PROCESSLIST;                          | All current connections and running queries |
| SHOW STATUS LIKE 'Threads_connected';      | Current number of connections               |
| SHOW VARIABLES LIKE 'max_connections';     | Maximum allowed connections                 |
| SHOW GLOBAL STATUS LIKE 'Threads_running'; | How many threads are actively running       |
| SHOW ENGINE INNODB STATUS\G                | InnoDB details, locks and long transactions |



### APPLICATION AND DATABASE EVIDENCE MUST BE CORRELATED

#### APPLICATION EVIDENCE



- Application logs (timeouts, connection errors)
- Connection pool metrics (active / idle)
- Response time spikes
- Error rates (5xx, 503, timeouts)
- Recent deployments or code changes

#### DATABASE EVIDENCE



- Number of connections
- Long running queries
- Locks and blocking
- CPU, memory and disk usage
- Slow query log

**WHEN BOTH SIDES AGREE, YOU WILL FIND THE REAL CAUSE.**



### INVESTIGATION FLOW



### JUNIOR ENGINEER TIPS

- Never assume the database is the only problem.
- Always check what changed before the incident.
- Look for patterns: time of day, specific feature, deployment.
- Ask: Are connections high, or are queries slow?
- Fix the cause so the problem does not return.



### REMEMBER THIS

- A HEALTHY DATABASE IS NOT ABOUT HIGH LIMITS. IT IS ABOUT STABLE, PREDICTABLE CONNECTIONS AND FAST QUERIES.



veriqta



veriqta



veriqta



veriqta



veriqta



veriqta



# CI/CD PIPELINE SUDDENLY FAILS



**BUILD WORKED YESTERDAY BUT FAILS TODAY.**

## 1. IDENTIFY THE EXACT FAILED STAGE



Click the failed stage. Read the logs. Understand the error. Do not guess. Do not rerun. Do not skip.

## 3. FOLLOW THIS DEBUGGING CHECKLIST

- ✓ Check the failed stage logs completely.
- ✓ Read the first error, not the last line.
- ✓ Check recent commits and configuration changes.
- ✓ Verify secrets, tokens, and credentials.
- ✓ Confirm dependencies and versions.
- ✓ Check runner / agent health and resources.
- ✓ Compare with the last successful run.
- ✓ Reproduce locally when possible.
- ✓ Document the root cause.

## 2. COMMON REASONS & WHAT TO CHECK

| REASON                       | WHAT IT MEANS                                                 | WHAT TO CHECK                                                     |
|------------------------------|---------------------------------------------------------------|-------------------------------------------------------------------|
| CREDENTIALS & EXPIRED TOKENS | Secrets, tokens, or keys are expired or revoked.              | Secrets store, token expiry, IAM permissions, service accounts.   |
| DEPENDENCY FAILURES          | Required packages or tools cannot be downloaded or found.     | Package registries, internet access, version changes, lock files. |
| RUNNER PROBLEMS              | The runner or agent is down, busy, or misconfigured.          | Runner status, logs, disk space, memory, concurrency limits.      |
| ENVIRONMENT DIFFERENCES      | Different OS, versions, variables, or configs cause failures. | Env vars, configs, OS versions, paths, time zones, tool versions. |
| CODE OR CONFIG CHANGES       | Recent changes introduced a bug or broken configuration.      | Commits, merge changes, config files, feature flags.              |

## 4. NEVER RERUN PIPELINES ENDLESSLY WITHOUT READING THE FAILURE



**RERUNNING WASTES TIME**

It doesn't fix the real problem.



**IT HIDES THE CAUSE**

The failure will come back.



**IT CONSUMES RESOURCES**

Runners, minutes and build credits.



**IT DELAYS EVERYONE**

The team waits longer for the same issue.



**READ. UNDERSTAND. THEN ACT.**

Fix the cause, not just the symptom.



## 5. QUICK COMMANDS & ACTIONS

| TOOL / COMMAND            | WHAT IT DOES                                   |
|---------------------------|------------------------------------------------|
| Check pipeline logs       | Read the output of the failed stage            |
| Check last successful run | Compare changes in behavior                    |
| Check secrets / variables | Verify token expiry and permissions            |
| Check dependencies        | Look for version conflicts or missing packages |
| Check runner status       | Ensure runner is online and healthy            |
| Reproduce locally         | Run the same build/test locally                |
| Check commit history      | Identify what changed recently                 |



## 6. GOOD PRACTICE REMINDERS

- ✓ Pin your tool and dependency versions.
- ✓ Use consistent environments across stages.
- ✓ Store secrets securely. Rotate tokens regularly.
- ✓ Fail fast with clear error messages.
- ✓ Add health checks for runners and services.
- ✓ Use caching wisely. Don't cache broken states.
- ✓ Monitor pipeline success rate and duration.
- ✓ Automate notifications for failures.



**REMEMBER**

**A PIPELINE FAILURE IS NOT RANDOM.  
THERE IS ALWAYS A CAUSE. FIND IT. FIX IT. PREVENT IT.**



veriqta



veriqta



veriqta



veriqta



veriqta



veriqta



# TLS CERTIFICATE INCIDENT

## ⚠️ USERS RECEIVE CERTIFICATE / SECURITY ERRORS

### COMMON CAUSES OF TLS CERTIFICATE INCIDENTS

#### 1 EXPIRED CERTIFICATE



The certificate has passed its validity period.

#### 2 WRONG HOSTNAME



The certificate does not match the domain users are visiting.

#### 3 BROKEN CERTIFICATE CHAIN



Missing intermediate certificates or untrusted certificate authority.

#### 4 CERTIFICATE ON WRONG ENDPOINT



The certificate is installed on the wrong server or load balancer.

#### 5 WEAK / UNSUPPORTED CONFIGURATION



Weak protocols, ciphers or deprecated TLS versions.

### 🔧 DIAGNOSTIC COMMANDS

#### 1. curl -v : QUICK CHECK FROM THE CLIENT SIDE

```
curl -v https://example.com
```

- Shows TLS handshake details.
- Reveals certificate subject, issuer, validity and errors.
- Look for lines like: **SSL certificate problem: certificate has expired or hostname mismatch.**

#### 2. openssl s\_client : DEEP TLS INSPECTION

```
openssl s_client -connect example.com:443 -servername example.com
```

- Shows the full certificate chain.
- Verifies expiration, issuer and trust.
- Look for: Verify return code: **0 (ok) = good**  
**non-zero = problem**

### 🔍 WHAT TO CHECK

| CHECK             | WHAT IT MEANS                                                       |
|-------------------|---------------------------------------------------------------------|
| EXPIRATION DATE   | Ensure the certificate is not expired.                              |
| HOSTNAME MATCH    | The domain in the certificate (CN/SAN) must match the URL.          |
| CERTIFICATE CHAIN | All intermediate certificates must be present and trusted.          |
| CORRECT ENDPOINT  | The certificate must be installed on the correct server / listener. |
| TLS CONFIGURATION | Use strong TLS versions (1.2 / 1.3) and secure ciphers.             |

### 🛡️ CERTIFICATE RENEWAL & MONITORING

- ✓ Automate certificate renewal (Let's Encrypt, ACM, cert-manager).
- ✓ Set up alerts for expiration (at least 30 days before).
- ✓ Monitor all domains, subdomains and environments.
- ✓ Track certificates in one inventory.
- ✓ Test renewal in staging before production.
- ✓ Verify renewal actually updated the live endpoint.



**RENEW EARLY, NOT LAST MINUTE. AUTOMATE AND MONITOR.**

### ★ WHY CERTIFICATE EXPIRY SHOULD BE DETECTED BEFORE USERS DO



Prevents user outages and loss of trust.



Avoids revenue loss and business impact.



Avoids emergency fixes under pressure.



Maintains security and compliance standards.



Planned renewals are safe and low risk.

### 👤 JUNIOR ENGINEER TIPS

- Never ignore certificate warnings.
- Always read the full error message.
- Verify the domain, chain and expiration.
- Use the diagnostic commands – don't guess.
- Fix the root cause, not just the symptom.

### ⚠️ COMMON TLS ERROR MESSAGES

- ERR\_CERT\_DATE\_INVALID → Certificate expired.
- ERR\_CERT\_COMMON\_NAME\_INVALID → Wrong hostname.
- ERR\_CERT\_AUTHORITY\_INVALID → Untrusted issuer / broken chain.
- ERR\_SSL\_PROTOCOL\_ERROR → Protocol mismatch or weak config.
- SEC\_ERROR\_UNKNOWN\_ISSUER → Missing intermediate certificate.



### REMEMBER

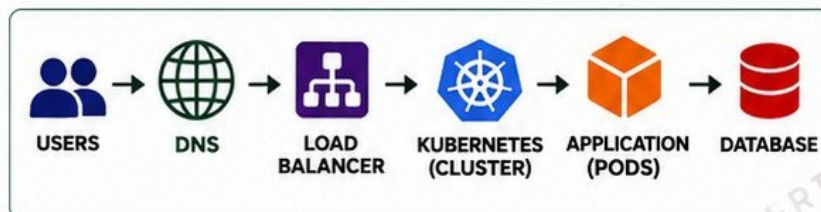
**CERTIFICATES PROTECT TRUST. MONITOR THEM LIKE ANY OTHER PRODUCTION SYSTEM.**



# FINAL PRODUCTION INCIDENT SIMULATION

**⚠️ REAL ENVIRONMENT. MULTIPLE SYMPTOMS. FIND THE ROOT CAUSE.**

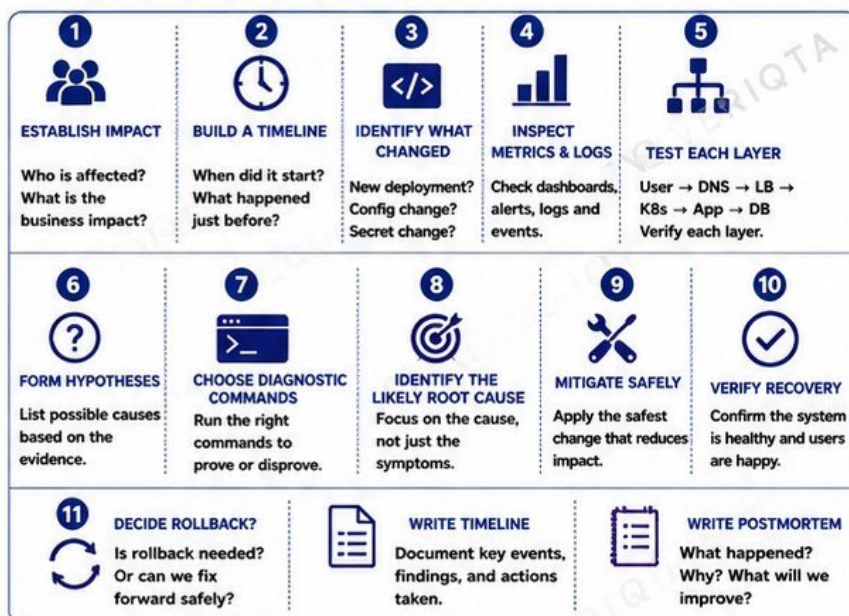
## 1. SYSTEM ARCHITECTURE



## 2. MULTIPLE SYMPTOMS

- ! Some users receive 502 Bad Gateway
- ⚙️ One Pod is CrashLoopBackOff
- 📈 CPU usage is rising on nodes
- 🗄️ Database connections are near the limit
- 🚀 A deployment happened 15 minutes ago
- 💔 One health check is failing

## 3. YOUR MISSION AS THE ENGINEER



## 4. USE THIS INVESTIGATION FLOW



## 5. STARTER CHECKLIST & COMMANDS BY LAYER

| LAYER         | WHAT TO CHECK                                   | EXAMPLE COMMANDS                                                                                                                             |
|---------------|-------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| USERS         | User reports, error messages, affected regions. | Check status page, monitoring, user feedback.                                                                                                |
| DNS           | DNS resolution, records, TTL, propagation.      | <code>dig example.com +short   nslookup example.com</code>                                                                                   |
| LOAD BALANCER | Health checks, target groups, 5xx errors.       | <code>curl -I https://your-domain.com   aws elbv2 describe-target-health ...</code>                                                          |
| KUBERNETES    | Pod status, events, logs, resource usage.       | <code>kubectl get pods -A   kubectl describe pod &lt;pod&gt;</code><br><code>kubectl logs &lt;pod&gt; --previous   kubectl get events</code> |
| APPLICATION   | App logs, errors, config, dependencies.         | <code>kubectl logs deploy/&lt;app&gt;   Check APM / error tracking</code>                                                                    |
| DATABASE      | Connections, queries, locks, CPU, slow queries. | <code>SHOW PROCESSLIST;   SELECT COUNT(*) FROM information_schema.PROCESSLIST;</code><br>Check DB dashboard / monitoring                     |



## 6. INCIDENT SUCCESS LOOKS LIKE THIS

- ✓ You reduced the impact quickly.
- ✓ You found and fixed the real root cause.
- ✓ You verified users are fully recovered.
- ✓ You documented what happened and what you learned.



## 7. REMEMBER

**INCIDENTS WILL HAPPEN.**  
**YOUR CALM PROCESS IS WHAT PROTECTS USERS.**  
**OBSERVE. THINK. TEST. FIX. VERIFY. LEARN.**